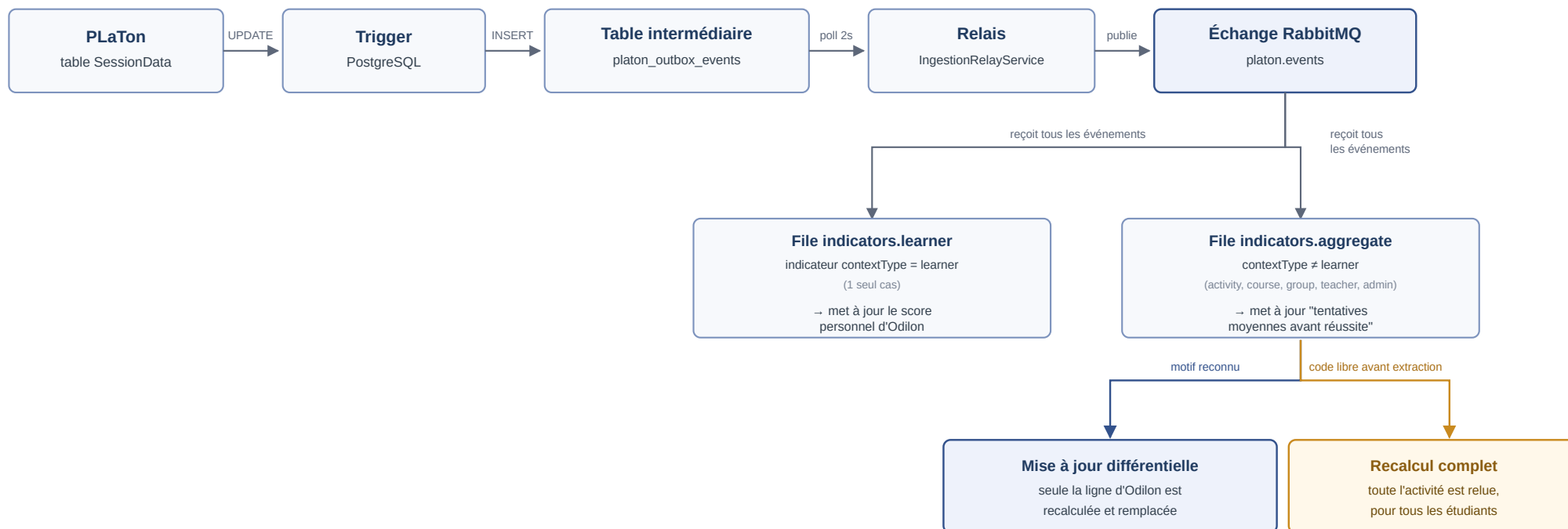


Pipeline d'ingestion des événements et calcul différentiel

Comment un événement PLaTon atteint un indicateur sans que PLaTon connaisse le microservice, et pourquoi la mise à jour n'a normalement pas besoin de tout relire.



Ce que montre le schéma : une même mise à jour (Odilon réussit un exercice à sa 4e tentative) part de PLaTon, traverse un trigger et une table intermédiaire posés sur la base de PLaTon (aucune modification du code de PLaTon), puis un relais qui republie vers RabbitMQ. Les deux files reçoivent chacune tous les événements et filtrent elles-mêmes ce qui les concerne : le tri se fait uniquement sur le **contextType de l'indicateur**, jamais sur le rôle de la personne connectée ni sur la complexité du calcul. "learner" est un seul cas précis (les données concernent un apprenant) ; tout le reste (y compris les indicateurs "teacher" et "admin") tombe dans "aggregate", pour que le calcul le plus lourd ne retarde jamais la mise à jour du score individuel. En bout de chaîne, seule une formule contenant une étape de code libre avant l'extraction de la valeur force une relecture complète de l'activité ; toutes les autres formules ne recalculent que la ligne modifiée.

Colonne source : add-platon-attempts-at-success-column.sql, trigger de maintenance sur Answers (indépendant du trigger d'ingestion ci-dessus)

Mise à jour différentielle : computeResultFromCandidates() / computeWithCandidateRows()

Trigger d'ingestion + table intermédiaire : event-rules.service.ts (DDL du trigger), platon_outbox_events

Deux files RabbitMQ : ingestion-consumer.service.ts, IngestionConsumerService, tri par contextType dans processAggregateIndicator()

Relais : ingestion-relay.service.ts, IngestionRelayService.relay(), @Cron toutes les 2s, classify()

Fork différentiel / complet : formula-interpreter.service.ts, getIncrementalShape() (null = recalcul complet, fallback interpret())